POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

SYLABUS PRZEDMIOTU

Nazwa przedmiotu:	**Programowanie aplikacji internetowych**
Kod przedmiotu:	PAI
Kierunek / Profil:	Informatyka / praktyczny
Tryb studiów:	stacjonarny
Rok / Semestr:	2 / 3
Charakter:	obowiązkowy
Odpowiedzialny:	mgr inż. Adam Urbanowicz
Wersja z dnia:	27.02.2026

1. Godziny zajęć i punkty ECTS

Wykłady	Ćwiczenia	Laboratori	Z prowa-dzącym	Praca własna	Łącznie	ECTS
30 h	—	60 h	90 h	60 h	150 h	**6**

2. Forma zajęć

Forma zajęć	Sposób zaliczenia
Wykład	Nieoceniany
Laboratorium	Zaliczenie z oceną

3. Cel dydaktyczny

Celem przedmiotu jest nabycie praktycznych umiejętności projektowania i implementowania nowoczesnych aplikacji internetowych z podziałem na backend i frontend, w oparciu o Domain-Driven Design (DDD) oraz architekturę oktogonalną. Student poznaje podstawy budowy rozwiązań typu SOA, mechanizmy uwierzytelniania i autoryzacji (OAuth 2.1), konteneryzację oraz praktyki wytwarzania oprogramowania obejmujące testy integracyjne

i testy end-to-end. Efektem jest umiejętność dostarczenia działającej aplikacji wraz z automatyzacją uruchamiania i testowania.

4. Przedmioty wprowadzające

Przedmiot	Wymagane zagadnienia
Podstawy programowania	swobodne tworzenie aplikacji w wybranym języku.
Relacyjne bazy danych	SQL, transakcje, relacje, podstawy modelowania danych
Technologie internetu	HTTP/HTTPS, REST, podstawy bezpieczeństwa, klient-serwer
Programowanie obiektowe	Podstawy OOP
Warsztat programisty	System kontroli wersji GIT. Dobra znajomość IDE programistycznego

5. Treści programowe

1. Wprowadzenie do aplikacji internetowych: backend vs frontend, kontrakty API, podstawy jakości i utrzymania.
2. DDD w praktyce: język ubiquitous, model domenowy, agregaty, invariants, repozytoria, domena vs aplikacja vs infrastruktura.
3. Architektura oktogonalna: porty i adaptery, granice odpowiedzialności, kierunek zależności, testowalność.
4. Wstęp do SOA: rozbicie na usługi, kontrakty, wersjonowanie API, spójność danych i integracja usług.
5. Bezpieczeństwo i autoryzacja: wprowadzenie do OAuth 2.1, tokeny, scenariusze integracyjne (w zakresie kursu).
6. Konteneryzacja: Docker, obrazy, sieci, wolumeny, docker-compose; uruchamianie pełnego środowiska projektu.
7. Budowa backendu: warstwy, API, walidacja, obsługa błędów, persystencja danych; integracja z innymi usługami (wg programu).
8. Budowa frontendu: podstawy architektury aplikacji klienckiej, integracja z API, zarządzanie stanem, obsługa błędów.
9. Testy integracyjne: strategia testów, testy kontraktów/warstw, uruchamianie z zależnościami (np. kontenery).
10. Testy end-to-end: scenariusze użytkownika, automatyzacja uruchomienia środowiska i testów.
11. Elementy procesu wytwarzania: Git flow / PR, code review, podstawy CI (pipeline uruchamiający testy).
12. Projekt zaliczeniowy: iteracyjna realizacja funkcji, dopięcie architektury, testów i uruchamiania w kontenerach.

6. Przedmiotowe efekty uczenia się

Wiedza

- Student zna i rozumie podstawowe pojęcia DDD (m.in. model domenowy, encje, wartości, agregaty, repozytoria, bounded context) oraz potrafi uzasadnić ich użycie.
- Student zna i rozumie założenia architektury oktogonalnej oraz zasady separacji warstw i zależności w systemie.
- Student zna i rozumie podstawy SOA oraz typowe style integracji usług (API, komunikacja synchroniczna/asynchroniczna – zależnie od programu zajęć).
- Student zna i rozumie mechanizmy uwierzytelniania i autoryzacji w aplikacjach internetowych, w tym OAuth 2.1 na poziomie koncepcji i przepływów.
- Student zna i rozumie rolę testów integracyjnych i end-to-end w cyklu wytwarzania oraz podstawy automatyzacji w CI.

Umiejętności

- Student potrafi zaprojektować i zaimplementować backend zgodny z DDD i architekturą oktogonalną, z czytelnym podziałem na warstwy oraz granice kontekstów.
- Student potrafi przygotować i zintegrować frontend z backendem poprzez API oraz zadbać o podstawowe aspekty bezpieczeństwa sesji/tokenów.
- Student potrafi zastosować OAuth 2.1 w aplikacji (konfiguracja i integracja po stronie aplikacji – zgodnie z założeniami kursu).
- Student potrafi przygotować środowisko uruchomieniowe w kontenerach (np. Docker + Compose), w tym zależności typu baza danych i usługi pomocnicze.
- Student potrafi napisać testy integracyjne dla kluczowych przypadków oraz testy E2E dla scenariuszy użytkownika i uruchamiać je automatycznie.

Kompetencje społeczne

- Student jest gotów do pracy zespołowej nad produktem programistycznym, w tym do komunikacji, podziału zadań i przeglądu kodu.
- Student jest gotów do odpowiedzialnego podejścia do jakości: testowania, utrzymania spójności architektury i dokumentowania rozwiązań.

7. Kryteria oceny

- Kolokwium (część wykładowa): pytania problemowe + zadania dotyczące DDD, architektury oktogonalnej, SOA oraz OAuth 2.1.
- Praca na zajęciach (laboratoria): realizacja zadań cząstkowych, jakość kodu, poprawność architektury, terminowość.
- Projekt: kompletność funkcjonalna, zgodność z założeniami (DDD + architektura oktogonalna), jakość testów (integracyjne + E2E), konteneryzacja i uruchamianie środowiska.
- Ocena końcowa = składowa z kolokwium + składowa z laboratoriów (praca) + składowa z projektu.

- Warunek zaliczenia: oddanie projektu oraz uzyskanie wyniku pozytywnego z kolokwium.

8. Metody dydaktyczne

wyklad:

- wykład problemowy
- analiza przypadków (case study) architektury systemów
- mini-demonstracje (live coding / przegląd repozytorium)

laboratorium:

- zadania praktyczne (warsztaty)
- praca zespołowa w repozytorium (code review, pull requesty)
- projekt semestralny iteracyjny (MVP -> kolejne przyrosty)
- testy automatyczne i pipeline CI

9. Literatura

Podstawowa:

- Eric Evans, "Domain-Driven Design: Tackling Complexity in the Heart of Software", 2003
- Vaughn Vernon, "Implementing Domain-Driven Design", 2013
- Robert C. Martin, "Clean Architecture", 2017

Uzupełniająca:

- Martin Fowler – artykuły o architekturze, testach, mikroserwisach i integracji.
- Dokumentacja OAuth 2.1 oraz dokumentacja narzędzi użytych na zajęciach (framework backend, framework frontend, Docker/Compose).
- Dokumentacja OpenAPI/Swagger (jeśli używane w projekcie).
- Materiały dotyczące testów E2E i CI/CD w wybranym stosie technologicznym.